Data-modeling process steps

This section details each step of the recommended data modeling process for Amazon DynamoDB.

Topics

- Step 1. Identify the use cases and logical data model
- Step 2. Create a preliminary cost estimation
- Step 3. Identify your data access patterns
- Step 4. Identify the technical requirements
- Step 5. Create the DynamoDB data model
- Step 6. Create the data queries
- Step 7. Validate the data model
- Step 8. Review the cost estimation
- Step 9. Deploy the data model

Step 1. Identify the use cases and logical data model

Objectives

- Gather the business needs and use cases that require a NoSQL database.
- Define the logical data model by using an entity-relationship (ER) diagram.

Process

- Business analysts interview business users to identify the use cases and the expected outcomes.
- Database engineer creates the conceptual data model.
- Database engineer creates the logical data model.
- Database engineer gathers information about item size, data volume, and expected read and write throughput.

Tools and resources

- Business requirements assessment (see template)

- Access patterns matrix (see template)
- Your preferred tool for creating diagrams

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
C	R/A	I	R		

Outputs

- Documented use cases and business requirements
- Logical data model (ER diagram)

Step 2. Create a preliminary cost estimation

Objective

- Develop a preliminary cost estimation for DynamoDB.

Process

- Database engineer creates the initial cost analysis using available information and the examples presented on the DynamoDB pricing page.
  - Create a cost estimate for on-demand capacity (see example).
  - Create a cost estimate for provisioned capacity (see example).
    - For the provisioned capacity model, get the estimate cost from the calculator and apply discount for reserved capacity.
  - Compare the estimated costs of the two capacity models.
  - Create an estimation for all the environments (Dev, Prod, QA).
- Business analyst reviews and approves or rejects the preliminary cost estimate.

Tools and resources

- AWS Pricing Calculator

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
C	A	I	R		

Outputs

- Preliminary cost estimation

Step 3. Identify your data access patterns

Access patterns or query patterns define how the users and the system access the data to satisfy business needs.

Objective

- Document the data access patterns.

Process

- Database engineer and business analyst interview the end users to identify how data will be queried using the data-access patterns matrix template.
  - For new applications, they review user stories about activities and objectives. They document the use cases and analyze the access patterns that the use cases require.
  - For existing applications, they analyze query logs to find out how people are currently using the system and to identify the key access patterns.
- Database engineer identifies the following properties of the access patterns:

  - Data size: Knowing how much data will be stored and requested at one time helps determine the most effective way to partition the data (see blog post).
  - Data shape: Instead of reshaping data when a query is processed (as an RDBMS system does), a NoSQL database organizes data so that its shape in the database corresponds with what will be queried. This is a key factor in increasing speed and scalability.
  - Data velocity: DynamoDB scales by increasing the number of physical partitions that are available to process queries, and by efficiently distributing data across those partitions. Knowing the peak query loads in advance might help determine how to partition data to best use I/O capacity.
- Business user prioritizes the access or query patterns.
  - Priority queries usually are the most used or most relevant queries. It is also important to identify queries that require lower response latency.

Tools and resources

- Access patterns matrix (see template)
- Choosing the Right DynamoDB Partition Key (AWS Database blog)
- NoSQL design for DynamoDB (DynamoDB documentation)

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
C	A	I	R		

Outputs

- Data-access patterns matrix

Example

Access pattern	Priority	Read or write	Description	Type (single item, multiple items, or all)	Key attribute	Filters	Result ordering
Create user profile	High	Write	User creates a new profile	Single item	Username	N/A	N/A
Update user profile	Medium	Write	User updates their profile	Single item	Username	Username = current user	N/A

Step 4. Identify the technical requirements

Objective

- Gather the technical requirements for the DynamoDB database.

Process

- Business analysts interview the business user and DevOps team to gather the technical requirements by using the assessment questionnaire.

Tools and resources

- Technical requirements assessment (see example questionnaire)

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
C	C	A	R		

Outputs

- Technical requirements document

Step 5. Create the DynamoDB data model

Objective

- Create the DynamoDB data model.

Process

- Database engineer identifies how many tables will be required for each use case. We recommend maintaining as few tables as possible in a DynamoDB application.
- Based on the most common access patterns, identify the primary key that can be one of two types: a primary key with a partition key that identifies data, or a primary key with a partition key and a sort key. A sort key is a secondary key for grouping and organizing data so it can be queried within a partition efficiently. You can use sort keys to define hierarchical relationships in your data that you can query at any level of the hierarchy (see blog post).
  - Partition key design
    - Define the partition key and evaluate its distribution.
    - Identify the need for write sharding to distribute workloads evenly.
  - Sort key design
    - Identify the sort key.
    - Identify the need for a composite sort key.
    - Identify the need for version control.

- Based on the access patterns, identify the secondary indexes to satisfy the query requirements.
  - Identify the need for local secondary indexes (LSIs). These are indexes that have the same partition key as the base table, but a different sort key.
    - For tables with LSIs, there is a 10 GB size limit per partition key value. A table with LSIs can store any number of items, as long as the total size for any one partition key value does not exceed 10 GB.
  - Identify the need for global secondary indexes (GSIs). These are indexes that have a partition key and a sort key that can be different from those on the base table (see blog post).
  - Define the index projections. Consider projecting fewer attributes to minimize the size of items written to the index. In this step, you should determine whether you want to use the following:
    - Sparse indexes
    - Materialized aggregation queries
    - GSI overloading
    - GSI sharding
    - An eventually consistent replica using GSI
- Database engineer determines whether the data will include large items. If so, they design the solution by using compression or by storing data in Amazon Simple Storage Service (Amazon S3).
- Database engineer determines whether time series data will be needed. If so, they use the time series design pattern to model the data.
- Database engineer determines whether the ER model includes many-to-many relationships. If so, they use an adjacency list design pattern to model the data.

Tools and resources

- NoSQL Workbench for Amazon DynamoDB — Provides data modeling, data visualization, and query development and testing features to help you design your DynamoDB database
- NoSQL design for DynamoDB (DynamoDB documentation)
- Choosing the Right DynamoDB Partition Key (AWS Database blog)
- Best practices for using secondary indexes in DynamoDB (DynamoDB documentation)
- How to design Amazon DynamoDB global secondary indexes (AWS Database blog)

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
I	I	I	R/A		

Outputs

- DynamoDB table schema that satisfies your access patterns and requirements

Example

The following screenshot shows NoSQL Workbench.

RetailDatabase | GSI: BundleProducts-VendorProducts-CustomerOrdersByOrderId | GSI: VendorOrdersByStatusDate-ProductInventory | GSI: ProductCatalog-CustomerOrdersByProduct

Primary Key		Attributes					
Partition Key: pk	Sort Key: sk						
P1	B1	GSI1-PK	GSI1-SK	name	desc		
		B1	P1	The Tiki Bundle	Everything you need for an island theme party.		
P4	B2	GSI1-PK	GSI1-SK	name	desc		
		B2	P4	Tiki Bar Set	Be the Mai Tai master with your very own Tiki Bar.		
P2	B1	name	desc	qty	GSI1-PK	GSI1-SK	location
		Tiki Torch	Bamboo tiki torch, 4 ft	6	B1	P2	W1-A9-S10-B52
	B2	name	desc	qty	GSI1-PK	GSI1-SK	location
		Tiki Torch	Bamboo tiki torch, 4 ft	2	B2	P2	W1-A9-S10-B52
	P2	name	desc	qty	location	reorderAt	GSI3-SK
		Tiki Torch	Bamboo tiki torch, 4 ft	656	W1-A9-S10-B52	100	/GardenOutdoor/OutdoorDecor/Lighting/LanternsT
	B1	name	desc	qty	GSI1-PK	GSI1-SK	location
		Tiki Statue - Pele	Tiki of the Hawaiian Fire Goddess Pele, 5 ft.	1	B1	P3	W1-A15-S6-B27

Step 6. Create the data queries

Objective

- Create the main queries to validate the data model.

Process

- Database engineer manually creates a DynamoDB table in the AWS Region or on their computer (DynamoDB Local).
- Database engineer adds sample data to the DynamoDB table.
- Database engineer builds facets using the NoSQL Workbench for Amazon DynamoDB or the AWS SDK for Java or Python to build sample queries (see blog post).

  Facets are like a view of the DynamoDB table.
- Database engineer and cloud developer build sample queries by using the AWS Command Line Interface (AWS CLI) or AWS SDK for the preferred language.

Tools and resources

- An active AWS account, to gain access to the DynamoDB console
- DynamoDB Local (optional), if you want to build the database on your computer without accessing the DynamoDB web service
- NoSQL Workbench for Amazon DynamoDB (download and documentation)
- AWS SDK in your choice of language (JavaScript, Python, PHP, .NET, Ruby, Java, Go, Node.js, C++, and SAP ABAP)

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
I	I	I	R/A	R	

Outputs

- Code to query the DynamoDB table

Examples

- DynamoDB examples using the AWS SDK for Java
- Python examples
- JavaScript examples

Step 7. Validate the data model

Objective

- Ensure that the data model will satisfy your requirements.

Process

- Database engineer populates the DynamoDB table with sample data.
- Database engineer runs the code to query the DynamoDB table.
- Database engineer collects the query results.
- Database engineer collects the query performance metrics.
- Business user validates that query results satisfy business needs.
- Business analysts validate the technical requirements.

Tools and resources

- An active AWS account, to gain access to the DynamoDB console
- DynamoDB Local (optional), if you want to build the database on your computer without accessing the DynamoDB web service
- AWS SDK in your choice of language

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
A	R	I	C		

Outputs

- Approved data model

Step 8. Review the cost estimation

Objectives

- Define the capacity model and estimate DynamoDB costs to refine the cost estimation from step 2.
- Get the final financial approval from the business analyst and stakeholders.

Process

- Database engineer identifies the data volume estimate.
- Database engineer identifies the data transfer requirements.
- Database engineer defines the required read and write capacity units.
- Business analyst decides between on-demand and provisioned capacity models.
- Database engineer identifies the need for DynamoDB auto scaling.
- Database engineer inputs the parameters in the Simple Monthly Calculator tool.
- Database engineer presents the final price estimation to business stakeholders.
- Business analyst and stakeholders approve or reject the solution.

Tools and resources

- AWS Pricing Calculator

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
C	A	I	R		

Outputs

- Capacity model
- Revised cost estimation

Step 9. Deploy the data model

Objective

- Deploy the DynamoDB table (or tables) to the AWS Region.

Process

- DevOps architect creates an CloudFormation template or other infrastructure as code (IaC) tool for the DynamoDB table (or tables). CloudFormation provides an automated way to provision and configure your tables and associated resources.

Tools and resources

- CloudFormation

RACI

Business user	Business analyst	Solutions architect	Database engineer	Application developer	DevOps engineer
I	I	C	C		R/A

Outputs

- AWS CloudFormation template

Example

```
mySecondDDBTable:
   Type: AWS::DynamoDB::
   Table DependsOn: "myFirstDDBTable"
   Properties:
   AttributeDefinitions:
      -  AttributeName: "ArtistId"
         AttributeType: "S"
      -  AttributeName: "Concert"
         AttributeType: "S"
      -  AttributeName: "TicketSales"
         AttributeType: "S"
   KeySchema:
      -  AttributeName: "ArtistId"
         KeyType: "HASH"
      -  AttributeName: "Concert"
         KeyType: "RANGE"
   ProvisionedThroughput:
      ReadCapacityUnits:
         Ref: "ReadCapacityUnits"
      WriteCapacityUnits:
         Ref: "WriteCapacityUnits"
   GlobalSecondaryIndexes:
      -  IndexName: "myGSI"
         KeySchema:
            - AttributeName: "TicketSales"
            KeyType: "HASH"
         Projection:
            ProjectionType: "KEYS_ONLY"
         ProvisionedThroughput:
         ReadCapacityUnits:
            Ref: "ReadCapacityUnits"
         WriteCapacityUnits:
            Ref: "WriteCapacityUnits"
   Tags:
      -  Key: mykey
```

```
        Value: myvalue
```